SBMS DEPLOYMENT AND OPERATIONS GUIDE

Prepared: 2026-11-02 **Revision described:** `fix/sbms-completion-work` @ `d6b5032` **Sources read:** `docs/azure-deployment-log-2026-10-31.md`, `docs/azure-deployment-plan-2026-10-30.md` §11-§12, `docs/deployment.md`, `docs/database-recovery-rehearsal-2026-10-31.md`, `docs/bursary-certificates-2026-11-01.md` §9, `Dockerfile`, `.github/workflows/publish-api-image.yml`, `render.yaml`.

---

1. Environments

Environment	Purpose	Runtime	Database	Status at this revision
Local development	Authoring and manual checks	Node 22 API on port 5000, Vite frontend on port 4174, Python 3.11 ML subprocess	Local PostgreSQL 17, database `student_beneficiary_system`	Working; the ML virtual environment must exist
Local disposable test	Automated suites	Jest and native runners against the same machine	`sbms_test` on loopback, role `sbms_test_runner`	Working; the guard refuses any other target
Deployment (demonstration)	Live end-to-end checks and the capstone demonstration	Azure Container Apps for the API, Azure App Service F1 for the frontend, Azure Database for PostgreSQL Flexible Server	`sbms` on `pg-sbms.postgres.database.azure.com`	Live and verified 2026-10-31, brought back up 2026-11-01
Render blueprint	Abandoned target	`render.yaml` remains in the repository	—	Superseded by Azure; kept only as history

The deployment is a **demonstration instance**, not a production service. It has one administrator path, demonstration data, no monitoring stack and no backup schedule. That description belongs in Chapter Five wherever the deployment is discussed.

2. Azure resource inventory

Resource	Name	Configuration	Purpose
Resource group	`rg-sbms`	UAE North	Container for everything below
Container Apps environment	`cae-sbms`	UAE North, logs destination none	Hosts the API container
Container app	`ca-sbms-api`	Image `ghcr.io/eggxellent/sbms-api`, `minReplicas 0`, public ingress	API plus ML runtime
App Service plan	`asp-sbms-web`	F1 (free), Linux	Hosts the frontend
Web app	`sbms-web`	F1, static build deployed as a zip	React single-page application
PostgreSQL flexible server	`pg-sbms`	Burstable B1MS, 32 GiB, stopped between demonstrations	`sbms` database
Budget	`budget-sbms`	Cost alert	Spend visibility

Verified facts that constrain any future change (`docs/azure-deployment-log-2026-10-31.md` §2): the subscription is Azure for Students with the spending limit ON, so no out-of-pocket spend is possible; a `sys.regionrestriction` policy denies every region except UAE North, Italy North, Austria East, Switzerland North and Spain Central; Static Web Apps cannot be created on this account because `staticSites` exists only in regions the policy denies; UAE North supports `Microsoft.App`, `Microsoft.DBforPostgreSQL`, `Microsoft.Web` and `Microsoft.ContainerRegistry`.

3. Live endpoints

Service	URL
API	`https://ca-sbms-api.ambitiousstone-f2de8f67.uaenorth.azurecontainerapps.io`
Frontend	`https://sbms-web.azurewebsites.net`
Database host	`pg-sbms.postgres.database.azure.com`, database `sbms`
Health	`GET /api/health` on the API host
Readiness	`GET /api/ready`, which also verifies the expected database name and required tables

4. Runtime architecture as deployed

Property	Value	Consequence
API and ML in one container	`duplicateCheck.js` spawns `predict.py --server` inside the same image	One deployable unit; no separate ML service, no network hop, no second authentication boundary
Image size	1.37 GB after cleanup (2.22 GB before)	Faster cold start; `deploy/` and `.env*` excluded from the image; the ML integrity gate runs inside the build
Image registry	Public `ghcr.io` package built by CI	No registry credential on the container app, and no Azure Container Registry cost
Scale	`minReplicas 0`	Scale to zero between demonstrations, which is why the API can take a cold start
Logs	Destination none	No Log Analytics workspace cost; the only diagnostics are the process's own stderr, which is why the start-up guard prints an authored cause
Frontend hosting	App Service F1, free tier	No cost; no staging slot; deployment is a zip push
Database	Burstable B1MS, stopped on demand	Cost only accrues while running; a stopped server makes the API unready by design
Secrets	`deploy/azure/.secrets.json`, git-ignored	Never printed in any document; rotating them is a manual step

5. Build and release pipeline

Stage	Mechanism	Evidence
API image build	GitHub Actions workflow `publish-api-image.yml`, dispatch only, registered on `dev`	Pushes `ghcr.io/eggxellent/sbms-api`
Build gates inside the image build	Dependency install, ML integrity gate	Recorded in the deployment log
Frontend build	`npm.cmd --prefix frontend run build` with `VITE_API_ORIGIN` set to the API origin	`dist/` written, 2 006 modules
Frontend deploy	`az webapp deploy` with a zip package	Live `index.html` served `assets/index-KTiUJ9pN.js` after the 2026-11-01 change
Migrations	**Manual.** An operator applies them; the pipeline never does	`backend/scripts/migrate.js` plus the backup-then-apply procedure
Local Docker	Docker CLI 29.9.2 present but the engine does not run in this environment	The image was therefore verified from the CI build, not locally

The repository being public is what makes CI minutes and `ghcr.io` packages free; that fact is recorded because it explains why no registry credential exists on the container app.

6. Configuration matrix

Variable	Required	Meaning	Failure if wrong
`DATABASE_URL`	yes	PostgreSQL connection string	API cannot start or reports not ready

Variable	Required	Meaning	Failure if wrong
`JWT_SECRET`	yes	>= 32 random bytes	Start-up aborts with `JWT_SECRET must contain at least 32 bytes and cannot be a placeholder.`
`PUBLIC_APP_URL`	yes	Public HTTPS frontend origin used in QR links	Start-up aborts; a wrong value produces unreachable QR codes
`CORS_ORIGINS`	yes in production	Bare frontend origin allowlist	Browser origins denied
`SBMS_EXPECTED_DATABASE_NAME`	yes in production	Database identity assertion	Readiness fails closed
`PORT`	no (default 5000)	Listen port	—
`NODE_ENV`	yes	`production` changes CORS and `PUBLIC_APP_URL` strictness	Wrong value changes security behaviour
`TRUST_PROXY_HOPS`	recommended behind a proxy	Number of trusted proxies	Unset behind App Service puts every user in one rate-limit bucket
`ACKNOWLEDGEMENT_OVERDUE_DAYS`	optional	1-3650	Unset disables overdue reporting
`ML_REVIEW_SCORE_THRESHOLD`	optional (0.50)	Review cut-off at start-up	A wrong value changes which pairs become cases
`DUPLICATE_HIGH_PRIORITY_THRESHOLD`	optional (0.70)	Presentation band	Changes ordering, not decisions
`DUPLICATE_REVIEW_THRESHOLD`	optional (0.30)	Legacy `standard` band	Cosmetic
`DUPLICATE_AGGREGATION_POLICY`	optional (`top-1`)	`top-1` or `all`	`all` reintroduces one case per pair
`VITE_API_ORIGIN`	frontend build time	API origin baked into the bundle	Frontend calls the wrong host

7. Start-up and readiness sequence

```
1. dotenv loads backend/.env (ignored by git)
2. JWT_SECRET is validated (length, placeholder, character diversity)
3. PUBLIC_APP_URL is resolved and validated (HTTPS in production)
4. Authentication schema is asserted: expected database name, public.users and its login columns
5. Express listens on PORT
6. GET /api/health answers {status: ok} immediately
7. GET /api/ready checks database identity and the required application tables, including
bursary_certificates
```

Any failure in steps 2-4 calls `process.exit(1)` with an authored sentence plus the cause. Steps 6 and 7 are deliberately separate: a live process is not a working system, and readiness fails closed rather than reporting ready because a socket is open. Neither endpoint initialises the database.

8. Database migration procedure

1. Read the SQL of every unapplied file in `backend/src/db/migrations/`.
2. Take a backup and verify it restores into a separate database.
3. Query the identity of the target (`current_user`, `current_database()`, host) and confirm it is the intended one.
4. Apply with `node backend/scripts/migrate.js`; the script records each file in `sbms_schema_migrations`.
5. Call `GET /api/ready` and require `{status: ready}` before announcing the deployment.
6. For the certificate feature specifically, `20261031_bursary_certificates.sql` must be applied before the endpoints are used; until then readiness reports not ready and issuing fails.

The application never applies a migration at start-up. That is a deliberate safety property: a container restart cannot change the schema.

9. Cost model (recorded, not re-derived)

Scenario	Cost
Idle month, database stopped, `minReplicas 0`	**$4.42** (PostgreSQL storage only: 32 GiB at $0.138/GiB-month)
One 8-hour demonstration	**$4.58** (adds about $0.16 of B1MS compute)
A normal month with occasional demonstrations	about **$6.42**
Frontend	$0 on App Service F1
Container registry	$0 (public `ghcr.io` package, no ACR)
Logs	$0 (logging destination none)

These figures come from `docs/azure-deployment-plan-2026-10-30.md` §12 and are owner decisions, not properties of the code. The spending limit on the subscription means the realistic worst case is a suspended service, not a bill.

10. Backup, restore and continuity

Property	State
Rehearsal performed	Yes, 2026-10-31, on the local disposable `sbms_test` database only
Method	`pg_dump -Fc` to a timestamped file outside the repository, then `pg_restore` into a newly created database whose name matched `^sbms_restore_rehearsal_\d{8}_\d{6}$`
Safety rules applied	Identity queried before any write; no overwrite of an existing backup; restore refused if the target name equalled `sbms_test`, `student_beneficiary_system` or `postgres`; no credential printed
Verification after restore	Readiness, table count, migration ledger and per-table row digests compared with the source
Backup file	Still present outside the repository; its hash is recorded in the rehearsal document
Scheduled backup in the deployment	**None**
Retention policy	**None**
Point-in-time recovery	Available as a platform feature of the managed service, never exercised

The rehearsal closes the "a documented, tested restore exists" criterion and nothing more. Continuity for a real service would need a schedule, a retention rule, an off-region copy and a restore drill with a stated recovery objective. None of those exist.

11. Operations runbook

Task	Command or action	Notes
Start the database	`az postgres flexible-server start --name pg-sbms --resource-group rg-sbms`	The API becomes ready once the database answers
Stop the database between demonstrations	`az postgres flexible-server stop --name pg-sbms --resource-group rg-sbms`	Stops accruing compute cost
Check liveness and readiness	`GET /api/health`, `GET /api/ready`	A 503 from readiness is a real fault, not a delay
Rebuild the API image	Dispatch `publish-api-image.yml` on `dev`, then update the container app	Dispatch only; nothing else is merged to `dev`
Deploy the frontend	`npm.cmd --prefix frontend run build` with `VITE_API_ORIGIN`, then `az webapp deploy` the zip	The live bundle name changes, which is the quick visual confirmation
Apply a migration	§8	Manual, after a verified backup
Rotate the JWT secret	Update the secret, restart the container app	Every existing token becomes invalid immediately

Task	Command or action	Notes
Reset an administrator password	Direct `UPDATE` with a freshly generated bcrypt hash, or create a new administrator through `POST /api/users` if any administrator can still sign in	Recorded as a live issue: the stored password drifted from the secret file (`docs/azure-deployment-log-2026-10-31.md` §9)

12. Known operational issues carried forward

Issue	State	What the report must say
Administrator password in `.secrets.json` no longer authenticates (`admin@sbms.com` and `admin@sbms.test` both 401)	Open	Administrator browser smoke checks are blocked until it is reset
Database was still running after the 2026-11-01 session, with a re-stop due by **2026-11-08 11:03Z**	Open at the time of the log	Cost accrues at $0.02/h until stopped
Two `E2E Probe` students and one duplicate case remain in the deployment	Open	The cleanup SQL is recorded in the deployment log
`frontend/src/pages/Verification.jsx` and `backend/src/index.js` were uncommitted in an earlier session	Resolved by later commits, but the log records the state at that time	Cite the commit, not the working tree
`browser-smoke.mjs` expects one shared `SBMS_VERIFY_PASSWORD`, so it cannot drive two accounts with different passwords	Open	Admin-side browser coverage is partial
Docker engine unavailable locally	Environmental	Image builds are evidenced from CI, not from a local build
Two root reports untracked; `.gitignore` lists the model artefact although it is tracked	Open (G-27, G-28)	Affects reproducibility from a clean clone

13. What the deployment does not demonstrate

Not demonstrated	Consequence
Load or concurrency behaviour	No hosted load test; the container scales to zero, so the first request after idle is a cold start
Monitoring, alerting or dashboards	Logs are disabled to save cost; there is no metrics review
Blue/green or canary release	A deploy is a single image swap with no rollback rehearsal
Multi-region or high availability	One region, one database instance, no replica
Backup schedule	§10
Real users	The instance holds demonstration data only
Phone-camera QR scanning against the public origin	Technically possible now that the origin is HTTPS, but no phone session was recorded (G-11)

Chapters Five and Six should describe this as an operated demonstration environment with verified functional behaviour on the dates recorded, and should not describe it as a production deployment.